PHASE 3: PROJECT DESIGN PHASE

3.1 Introduction

The Project Design Phase defines how the **Implement Client Script & UI Policy (Incident)** project will be structured in ServiceNow. This phase converts the requirements identified in Phase 2 into a clear design for UI Policies, UI Policy Actions, Client Scripts, and validation rules.

The design focuses on controlling Incident fields based on the **Impact** value, automating required field values, validating information before submission, and controlling list-based updates.

3.2 Design Objectives

The main objectives of the project design are:

- To define the required UI Policy and Client Script configurations.
- To control Incident fields based on specific conditions.
- To automate the Urgency value when Impact is High.
- To ensure required Incident information is provided.
- To prevent invalid Incident submissions.
- To control State changes through list editing.
- To provide clear feedback to users.
- To maintain accurate and consistent Incident records.

3.3 Incident Form Design

The project is designed around the **ServiceNow Incident form**. The main fields involved in the design are:

- **Impact**
- **Assignment Group**
- **Urgency**
- **Assigned To**
- **State**

The **Impact** field acts as the main condition for several controls. When Impact is set to High, additional rules are applied to the Incident form.

The Assignment Group and Urgency fields are controlled through the UI Policy, while Assigned To and State are handled through Client Scripts.

This design ensures that different Incident fields have clearly defined responsibilities.

3.4 UI Policy Design

A UI Policy named **High Impact Control** will be created for the **Incident** table.

The policy will use the following condition:

Impact = 1 – High

When the condition is satisfied:

- **Assignment Group** will become mandatory.
- **Urgency** will be controlled using a UI Policy Action.
- **Reverse if False** will be enabled.

The Reverse if False option ensures that the field behavior returns to normal when Impact is changed from High to another value.

The UI Policy therefore provides dynamic control of the Incident form based on the Impact value.

3.5 UI Policy Action Design

A UI Policy Action will be created under the **High Impact Control** policy to control the **Urgency** field.

The configuration will be:

Property	Configuration
Field	Urgency

Read-only	True
Visible	Unchanged
Applied when	UI Policy condition is satisfied

When the Impact condition is satisfied, the Urgency field will remain visible but the user will not be able to modify it.

When the condition becomes false, the **Reverse if False** setting will restore the normal behavior of the field.

3.6 Client Script Design

Three Client Scripts will be designed for different Incident events.

1. onChange Client Script

The onChange Client Script will be associated with the **Impact** field. When the user changes Impact to High, the script will automatically set **Urgency** to High.

An informational message will also be displayed to inform the user that Urgency has been updated automatically.

2. onSubmit Client Script

The onSubmit Client Script will run when the user attempts to save the Incident.

The script will check whether:

- Impact is High.
- Assigned To is empty.

If both conditions are satisfied, the Incident will not be saved and an error message will be displayed for the Assigned To field.

3. onCellEdit Client Script

The onCellEdit Client Script will be associated with the **State** field.

When a user attempts to change State directly from the Incident list, the script will display an alert and prevent the change.

State changes through the normal Incident form will remain available.

3.7 Validation and Control Design

The final design includes validation and control mechanisms to ensure that Incident records follow the defined requirements.

Field / Area	Design Control
Impact	Acts as the main condition for UI Policy behavior
Assignment Group	Mandatory when Impact is High
Urgency	Read-only when Impact is High
Urgency Update	Automatically set to High when Impact becomes High
Assigned To	Required before saving a High-impact Incident
State – List	Direct editing is prevented
State – Form	Normal form-based changes are allowed

The design also includes appropriate user feedback. Informational messages are used for automatic updates, while error and alert messages are used when an action is not permitted.

Overall, this design provides a clear structure for implementing the required **UI Policies, UI Policy Actions, and Client Scripts** while maintaining consistent and accurate Incident data.